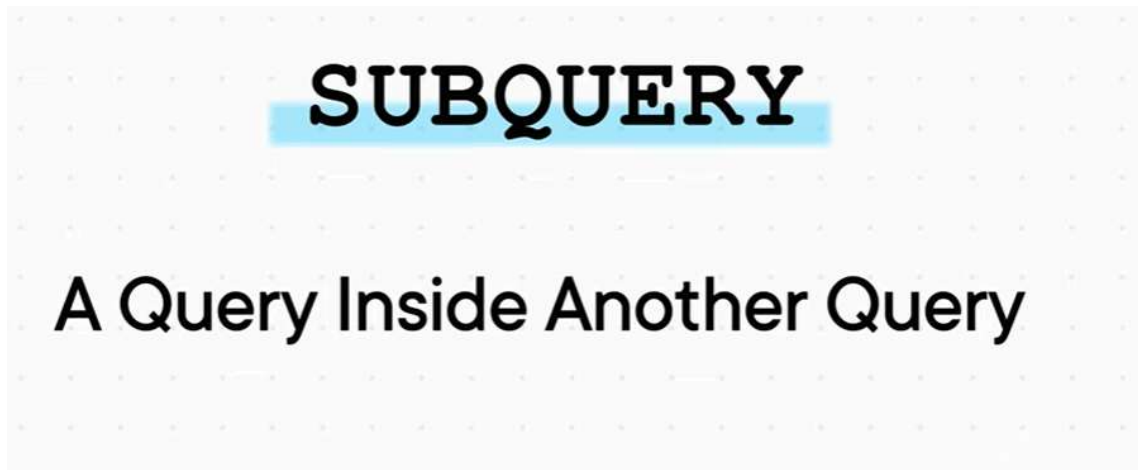


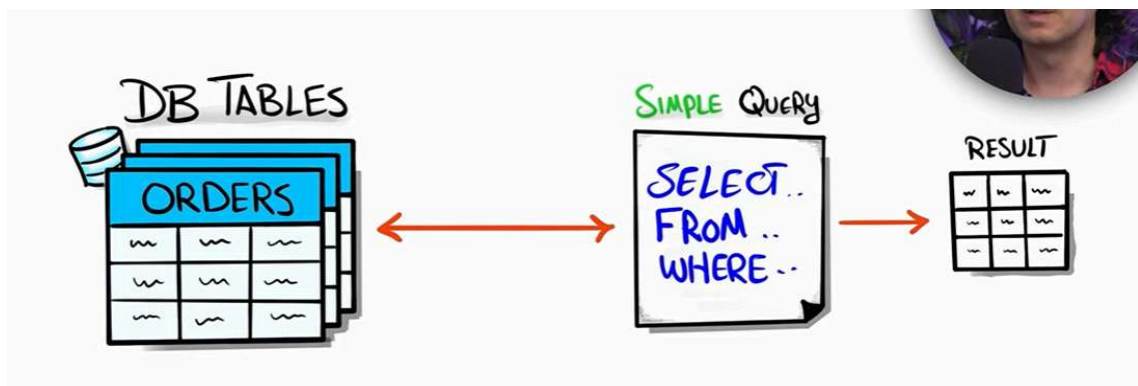
SQL Subqueries 👍

We will learn diff types of subqueries and operators used
And comparison IN, ANY, Exists ,ALL etc

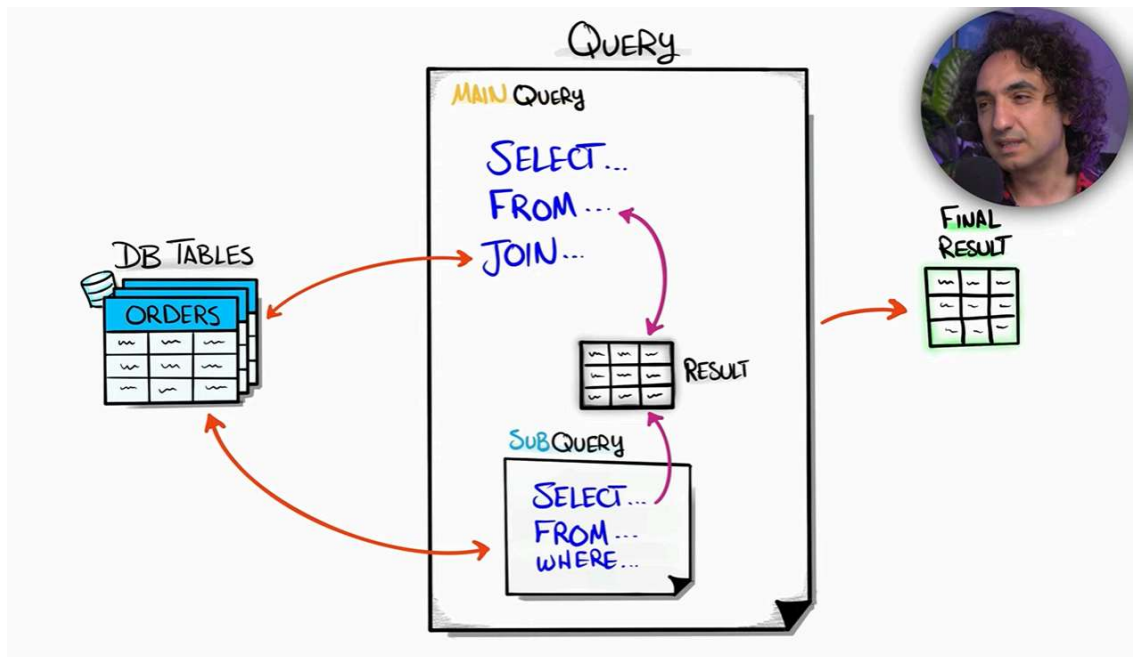
How sql execute subquery learn step by step -



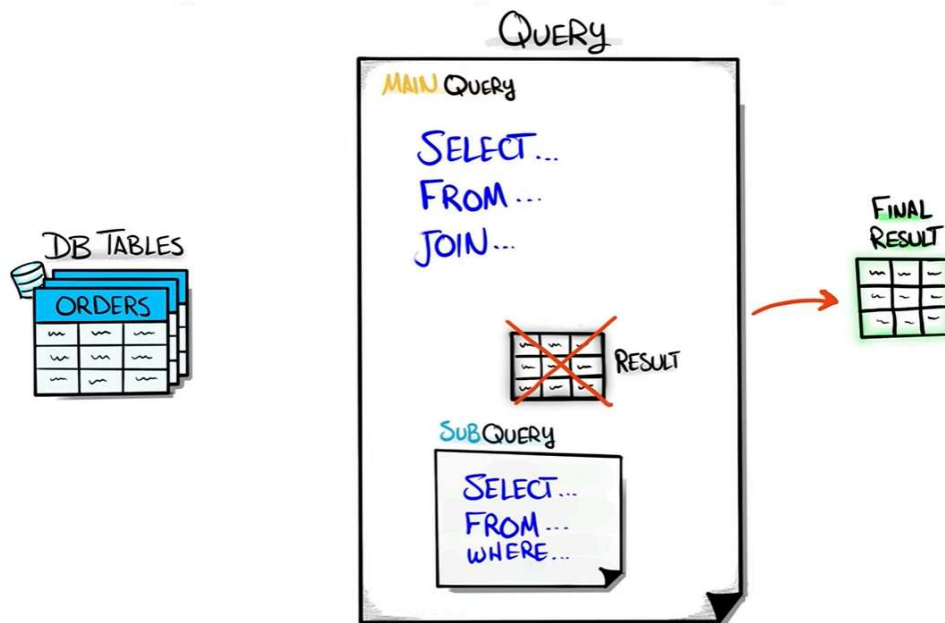
Very simple query -



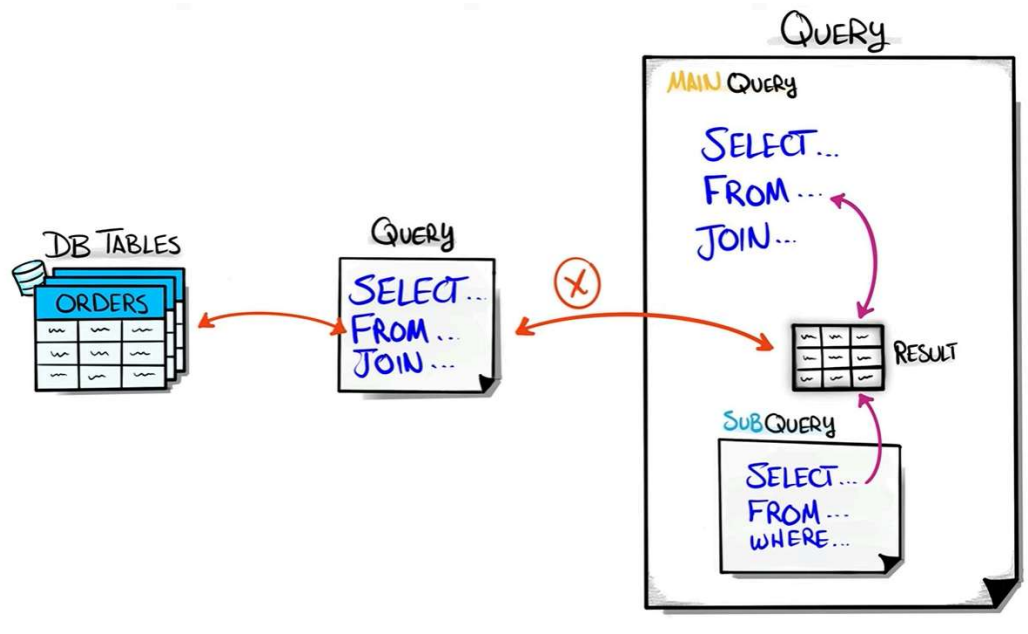
Subquery - It play role of supporter.



Once the execution of query completed or done. Sql will delete /destroy result of subquery.



Intermediate result of subquery only locally mean to the main query only. It is not available for any external or other query. Below is example



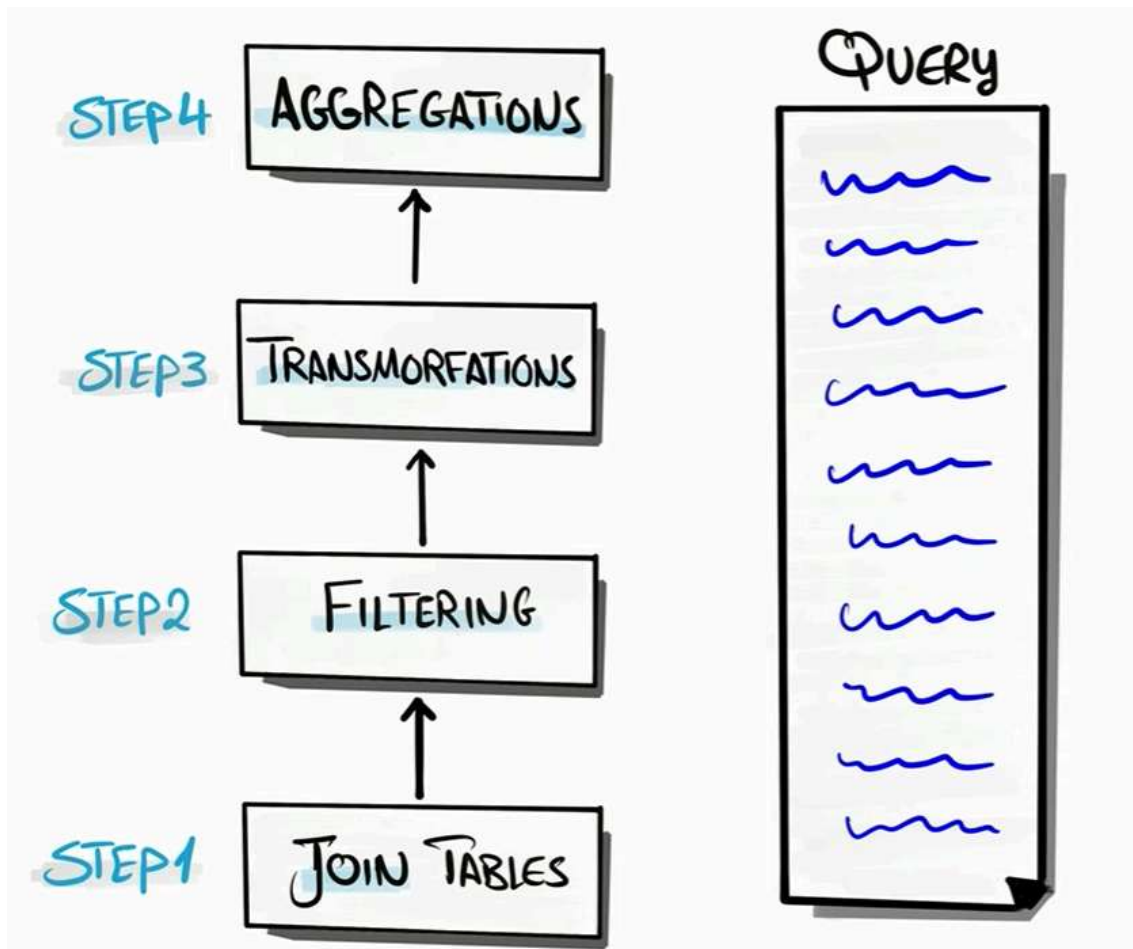
rs AS (
Sales
rs
ID

- When to use -

WHY SUBQUERIES?

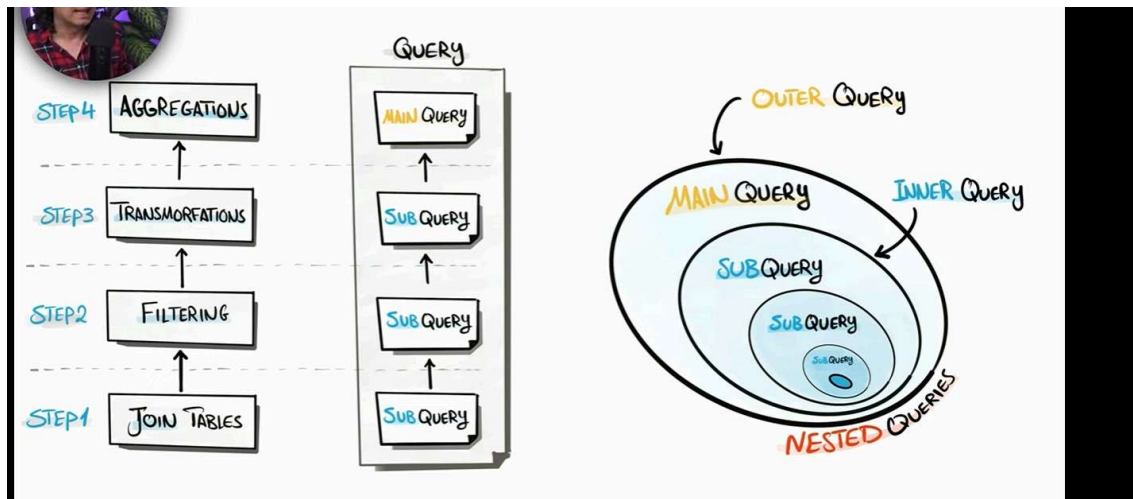
acts p
tOrders cte

CASE WHEN Sales > 50 THEN



this is not correct single query for all steps query will be large and complex.

So make query for each step like preparation of one step to the next step.

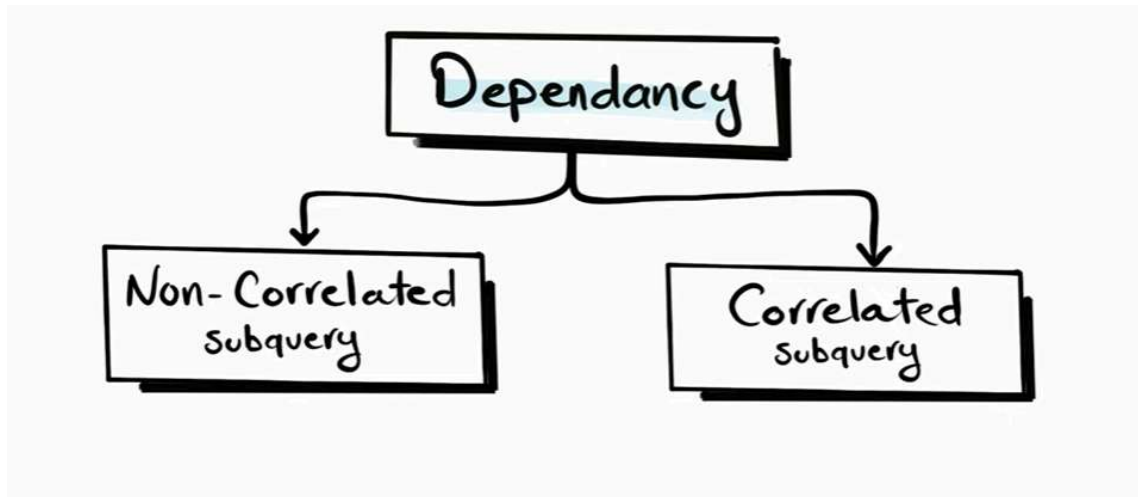


Subquery

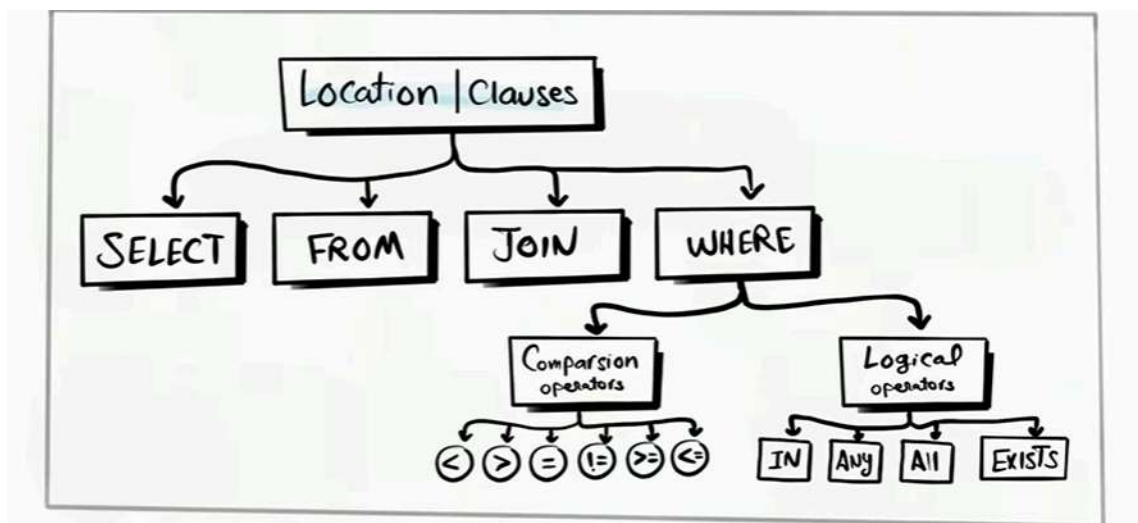
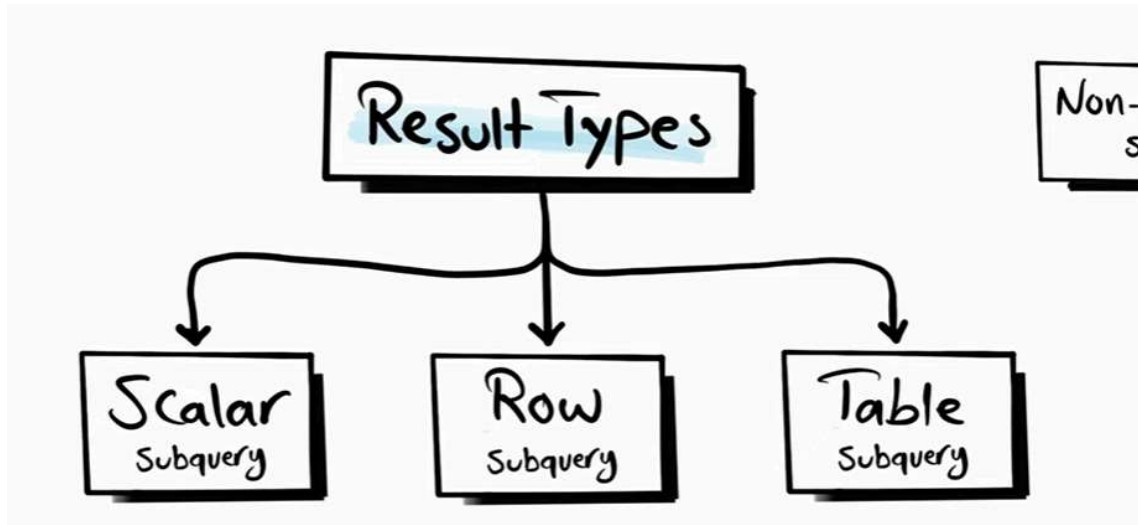
Categories

Dependency of subquery -

1. Non - correlated subquery means subquery not depend on main query
2. Correlated sub query means subquery depend on the main query.



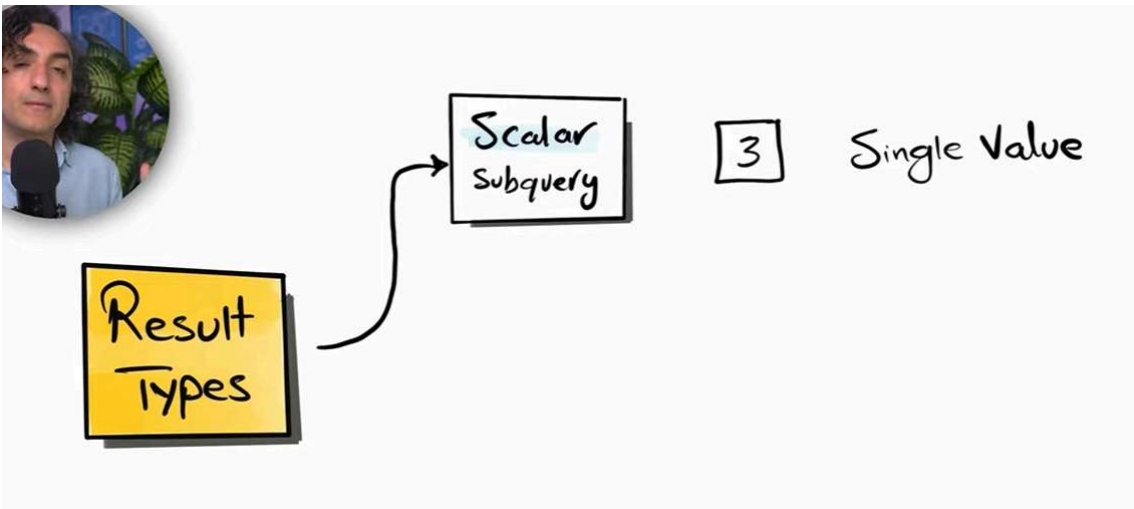
Subquery divided based on result type -



Subq

Result Types

Amount of data that subquery is going to return.



A screenshot of the SQL Server Enterprise Manager interface. The query editor shows the following SQL query:

```
SELECT *  
FROM Sales.Orders
```

The results pane displays a table with 10 rows and 9 columns. The columns are: OrderID, ProductID, CustomerID, SalesPersonID, OrderDate, ShipDate, OrderStatus, ShipAddress, and a truncated column. The data is as follows:

	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	
1	1	101	2	3	2025-01-01	2025-01-05	Delivered	9833 Mt. Dias Blv.	1
2	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	N
3	3	101	1	5	2025-01-10	2025-01-25	Delivered	8157 W. Book	8
4	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane	
5	5	104	2	5	2025-02-01	2025-02-05	Delivered	NULL	N
6	6	104	3	5	2025-02-05	2025-02-10	Delivered	1792 Belmont Rd.	N
7	7	102	1	1	2025-02-15	2025-02-27	Delivered	136 Balboa Court	
8	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4
9	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way	
10	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	N

SQLQuery1.sql - D:\8QBU\Youtube (29) |

```
SELECT  
    AVG(Sales)  
FROM Sales.Orders
```

245 %

Results Messages

	(No column name)
1	38

↑

SCALAR QUERY



SQLQuery1.sql - D:\8QBU\Youtube (59) |

```
SELECT  
    CustomerID  
FROM Sales.Orders
```

245 %

Results Messages

	CustomerID
1	2
2	3
3	1
4	1
5	2
6	3
7	1
8	4
9	2
10	3

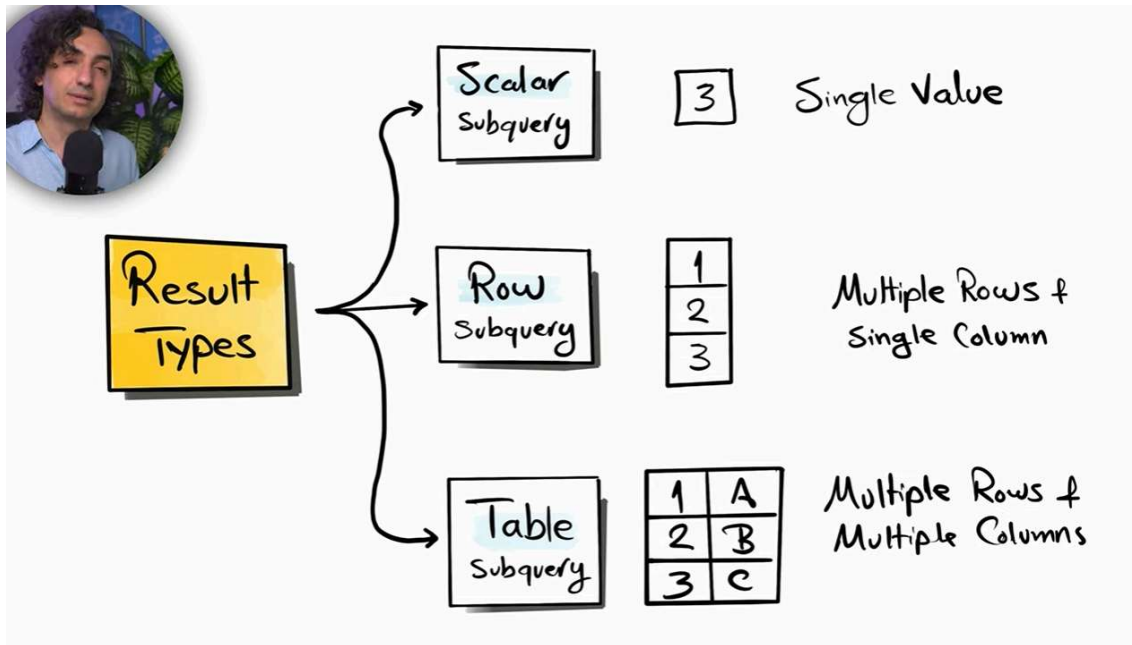
←

ROW QUERY



Query executed successfully.

DESKTOP-84B80BL\SQLEXPRESS ... DESKTOP-84B80BL\You



SQLQuery1.sql - D:\8QBU\Youtube (59) -

```
SELECT  
  OrderID,  
  OrderDate  
FROM Sales.Orders
```

Results

	OrderID	OrderDate
1	1	2025-01-01
2	2	2025-01-05
3	3	2025-01-10
4	4	2025-01-20
5	5	2025-02-01
6	6	2025-02-05
7	7	2025-02-15
8	8	2025-02-18
9	9	2025-03-10
10	10	2025-03-15

← **TABLE QUERY**

Query executed successfully.

DESKTOP-84B8QBU\SQLEXPRESS ... DESKTOP-84B8QBU\Youtub.

Subquery in FROM Clause

```
ers cte  
ProductID
```

```
CASE WHEN Sales  
      WHEN Sales  
      ELSE 'Low'
```

FROM Subquery

Used as tempoorary table for the main query



Subquery in
FROM Clause

```
SELECT column1, column2,...  
FROM ( SELECT column FROM table1 WHERE condition ) AS alias
```

Main Query

Subquery

Sql ^

 Copy

```
SELECT category, avg(price) as avg_price
FROM products
GROUP BY category;
```

Result:

category	avg_price
Electronics	150
Furniture	350
Grocery	50

```
SELECT product_id, category, price,
       avg(price) over () as avg_price
FROM products;
```

Result:

product_id	category	price	avg_price
1	Electronics	100	210
2	Electronics	200	210
3	Furniture	300	210
4	Furniture	400	210
5	Grocery	50	210

SQLQuery8.sql - D:\8QBU\Youtube (52)* SQLQuery7.sql - D:\8QBU\Youtube (53)*

```
-- Find the products that have a price higher than the average price of all products

--Main Query
SELECT
*
FROM
-- Subquery
(SELECT
ProductID,
Price,
AVG(Price) OVER () AvgPrice
FROM Sales.Products)t
WHERE price > AvgPrice
```

184 %

TIP ProductID Price AvgPrice

To check the intermediate results of a subquery, highlight it and execute



Execute highlighted intermediate result or sub query result for testing.

Executed only highlighted query

SQLQuery8.sql - D:\8QBU\Youtube (52)* SQLQuery7.sql - D:\8QBU\Youtube (53)*

```
-- Find the products that have a price higher than the average price of all products

--Main Query
SELECT
*
FROM
-- Subquery
(SELECT
ProductID,
Price,
AVG(Price) OVER () AvgPrice
FROM Sales.Products)t
WHERE price > AvgPrice
```

184 %

Results Messages

	ProductID	Price	AvgPrice
1	101	10	20
2	102	15	20
3	103	20	20
4	104	25	20
5	105	30	20

Query executed successfully.

DESKTOP-B4B8QBU\SQLEXPRESS ... DESKTO



SQLQuery8.sql - D:\8QBU\Youtube (52)* SQLQuery7.sql - D:\8QBU\Youtube (53)*

```
-- Find the products that have a price higher than the average price of all products  
--Main Query  
SELECT  
*  
FROM  
    -- Subquery  
    (SELECT  
        ProductID,  
        Price,  
        AVG(Price) OVER () AvgPrice  
        FROM Sales.Products)t  
WHERE price > AvgPrice
```



184 %

	ProductID	Price	AvgPrice
1	104	25	20
2	105	30	20

SQLQuery8.sql - D:\8QBU\Youtube (52)* SQLQuery5.sql - D:\8QBU\Youtube (55)*

```
-- Rank Customers based on their total amount of sales  
-- Main Query  
SELECT  
*,  
RANK() OVER (ORDER BY TotalSales DESC) CustomerRank  
FROM  
    -- Subquery  
    (SELECT  
        CustomerID,  
        SUM(Sales) TotalSales  
        FROM Sales.Orders  
        GROUP BY CustomerID)t
```

184 %

	CustomerID	TotalSales	CustomerRank
1	3	125	1
2	1	110	2
3	4	90	3
4	2	55	4

How sql is going to execute subquery

Orders		
OrderID	CustomerID	Sales
1	2	10
2	3	15
3	1	20
4	1	60
5	2	25
6	3	50
7	1	60
8	4	90
9	2	20
10	3	60



Subquery

Original Query

```
SELECT
*,
RANK() OVER (ORDER BY TotalSales DESC) CustomerRank
FROM
(SELECT
CustomerID,
SUM(Sales) TotalSales
FROM Sales.Orders
GROUP BY CustomerID)t
```



Orders		
OrderID	CustomerID	Sales
1	2	10
2	3	15
3	1	20
4	1	60
5	2	25
6	3	50
7	1	60
8	4	90
9	2	20
10	3	60



Subquery

Original Query

```
SELECT
*,
RANK() OVER (ORDER BY TotalSales DESC) CustomerRank
FROM
(SELECT
CustomerID,
SUM(Sales) TotalSales
FROM Sales.Orders
GROUP BY CustomerID)t
```

```
SELECT
CustomerID,
SUM(Sales) TotalSales
FROM Sales.Orders
GROUP BY CustomerID
```



Subquery

```
SELECT
  CustomerID,
  SUM(Sales) TotalSales
FROM Sales.Orders
GROUP BY CustomerID
```

CustomerID	TotalSales
1	110
2	55
3	125
4	90

Main Query

```
SELECT
  *,
  RANK() OVER (ORDER BY TotalSales DESC) R
FROM
```

1	110
2	55
3	125
4	90

Main Query

```
SELECT
  *,
  RANK() OVER (ORDER BY TotalSales DESC) R
FROM
```

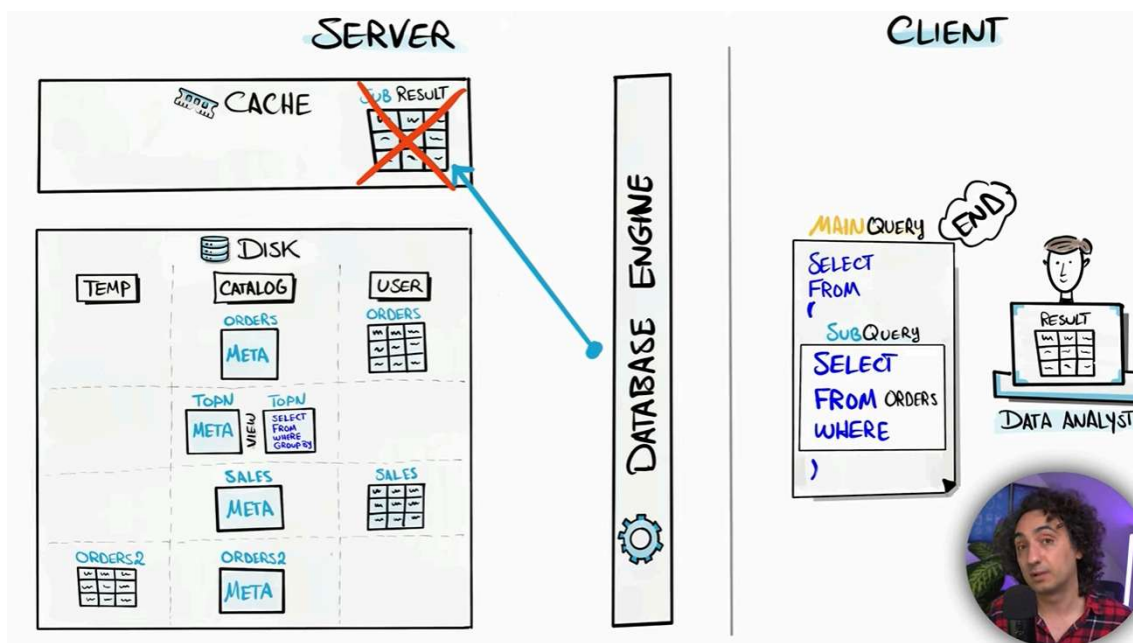
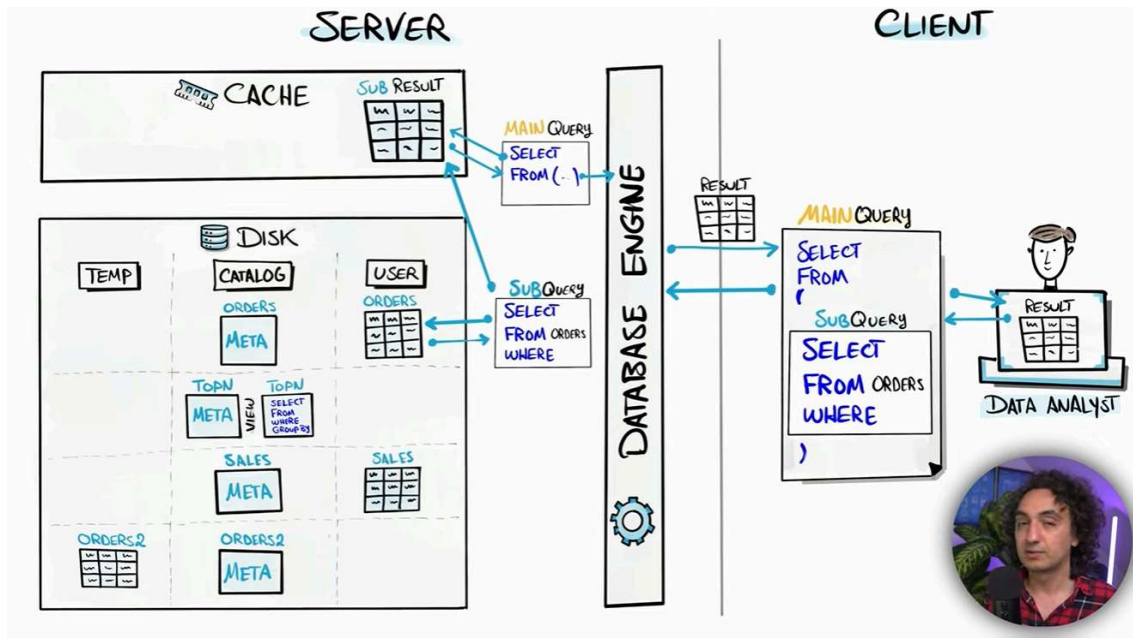
FINAL
RESULT

CustomerID	TotalSales	R
3	125	1
1	110	2
4	90	3
2	55	4



How Database

Execute Subqueries



Subquery in **SELECT Clause**

SELECT Subquery

Used to aggregate data side by side with the main query's data, allowing for direct comparison.



Subquery in
SELECT Clause

```
SELECT  
  Column1,  
  ( SELECT column FROM table1 WHERE condition ) AS alias  
FROM table1
```

Main Query

Subquery

Rules

Only Scalar Subqueries are allowed to be used

SQLQuery8.sql - D:\8QBU\Youtube (52)* SQLQuery7.sql - D:\8QBU\Youtube (53)*

```
-- Show the product IDs, product names, prices, and the total number of orders

-- Main Query
SELECT
    ProductID,
    Product,
    Price,
    -- Subquery
    (SELECT COUNT(*) FROM Sales.Orders) AS TotalOrders
FROM Sales.Products
```

184 %

Results Messages

	ProductID	Product	Price	TotalOrders
1	101	Bottle	10	10
2	102	Tire	15	10
3	103	Socks	20	10
4	104	Caps	25	10
5	105	Gloves	30	10



SQLQuery8.sql - D:\8QBU\Youtube (52)* SQLQuery7.sql - D:\8QBU\Youtube (53)*

```
-- Show the product IDs, product names, prices, and the total number of orders

-- Main Query
SELECT
    ProductID,
    Product,
    Price,
    -- Subquery
    (SELECT OrderID FROM Sales.Orders) AS TotalOrders
FROM Sales.Products
```

94 %

Results Messages

Msg 512, Level 16, State 1, Line 4
Subquery returned more than 1 value. This is not permitted when the subquery follows a SELECT list.

Completion time: 2024-07-28T21:18:54.5873265+02:00



How the subquery executed

Products		
ProductID	Product	Price
101	Bottle	10
102	Tire	15
103	Socks	20
104	Caps	25
105	Gloves	30

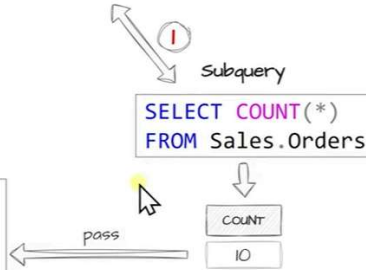
Orders		
OrderID	CustomerID	Sales
1	2	10
2	3	15
3	1	20
4	1	60
5	2	25
6	3	50
7	1	60
8	4	90
9	2	20
10	3	60

Original Query

```
SELECT
  ProductID,
  Product,
  (SELECT COUNT(*)
   FROM Sales.Orders) Total
FROM Sales.Products
```

Main Query

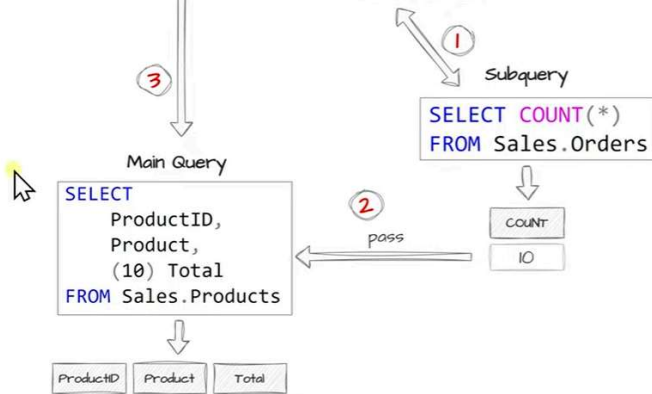
```
SELECT
  ProductID,
  Product,
  (10) Total
FROM Sales.Products
```

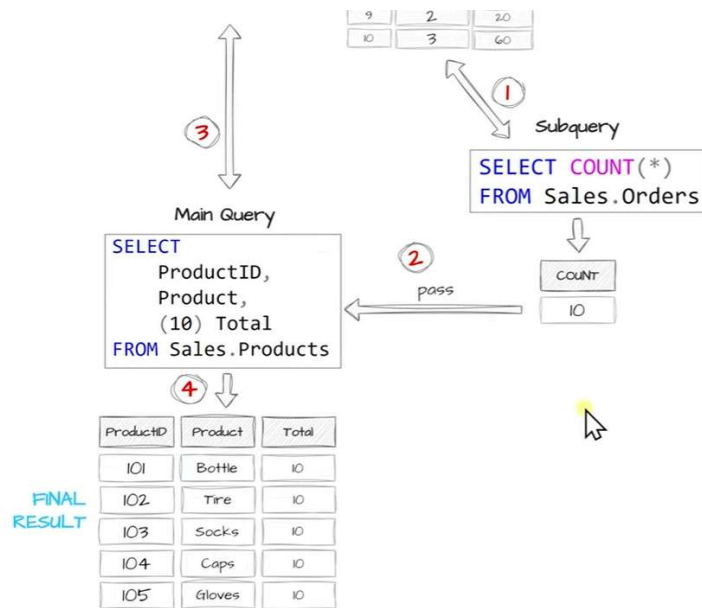


8	4	90
9	2	20
10	3	60

Main Query

```
SELECT
  ProductID,
  Product,
  (10) Total
FROM Sales.Products
```





How to use subquery in joining clause

Subquery in JOIN Clause

JOIN Subquery

Used to prepare the data (filtering or aggregation) before joining it with other tables.

SQLQuery3.sql - D:\8QBUI\Youtube (68) - SQLQuery2.sql - D:\8QBUI\Youtube (70) - SQLQuery1.sql - D:\8QBUI\Youtube (81)

```
-- Show all customer details and find the total orders of each customer
```

```
SELECT
*
FROM Sales.Customers;

SELECT
CustomerID,
COUNT(*) TotalOrders
FROM Sales.Orders
GROUP BY CustomerID;
```

183 %

	CustomerID	FirstName	LastName	Country	Score
1	1	Jossef	Goldberg	Germany	350
2	2	Kevin	Brown	USA	900
3	3	Mary	NULL	USA	750
4	4	Mark	Schwarz	Germany	500
5	5	Anna	Adams	USA	NULL

	CustomerID	TotalOrders
1	1	3
2	2	3
3	3	3
4	4	1

Query executed successfully. DESKTOP-84B8QBUI\SQLEXPRESS



SQLQuery3.sql - D:\8QBUI\Youtube (68) - SQLQuery2.sql - D:\8QBUI\Youtube (70) - SQLQuery1.sql - D:\8QBUI\Youtube (81)

```
-- Show all customer details and find the total orders of each customer
```

```
--Main Query
SELECT
*
FROM Sales.Customers c
LEFT JOIN (
SELECT
CustomerID,
COUNT(*) TotalOrders
FROM Sales.Orders
GROUP BY CustomerID) o
ON c.CustomerID = o.CustomerID
```

151 %

	CustomerID	FirstName	LastName	Country	Score	CustomerID	TotalOrders
1	1	Jossef	Goldberg	Germany	350	1	3
2	2	Kevin	Brown	USA	900	2	3
3	3	Mary	NULL	USA	750	3	3
4	4	Mark	Schwarz	Germany	500	4	1
5	5	Anna	Adams	USA	NULL	NULL	NULL



SQLQuery3.sql - D:\8Q8U\Youtube (68)) * SQLQuery2.sql - D:\8Q8U\Youtube (70)) * SQLQuery1.sql - D:\8Q8U\Youtube (81)) *

```
-- Show all customer details and find the total orders of each customer

--Main Query
SELECT
  c.*,
  o.TotalOrders
FROM Sales.Customers c
LEFT JOIN (
  SELECT
    CustomerID,
    COUNT(*) TotalOrders
  FROM Sales.Orders
  GROUP BY CustomerID) o
ON c.CustomerID = o.CustomerID
```

151 %

Results Messages

	CustomerID	FirstName	LastName	Country	Score	TotalOrders
1	1	Jossef	Goldberg	Germany	350	3
2	2	Kevin	Brown	USA	900	3
3	3	Mary	NULL	USA	750	3
4	4	Mark	Schwarz	Germany	500	1
5	5	Anna	Adams	USA	NULL	NULL

Subquery in WHERE Clause

```
cte
productID
CASE WHEN Sales > 50 THEN
WHEN Sales > 20 THEN
```

WHERE Subquery

Used for complex filtering logic and makes query more flexible and dynamic.

COMPARISON OPERATORS

Used to filter data by comparing two values



Comparison Operators

=	Equal	WHERE Sales = (SELECT AVG(Sales) FROM ORDERS)
!= <>	Not Equal	WHERE Sales != (SELECT AVG(Sales) FROM ORDERS)
>	Greater than	WHERE Sales > (SELECT AVG(Sales) FROM ORDERS)
<	Less than	WHERE Sales < (SELECT AVG(Sales) FROM ORDERS)
>=	Greater than or equal to	WHERE Sales >= (SELECT AVG(Sales) FROM ORDERS)
<=	Less than or equal to	WHERE Sales <= (SELECT AVG(Sales) FROM ORDERS)